



CSE 3313: Computer Architecture

Sidratul Tanzila Tasmi

Lecturer, Department of Computer Science and Engineering
United International University

	Instruction	Example	Meaning	Comments
	add	add \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$	three register operands
	subtract	sub \$s1,\$s2,\$s3	$\$s1 = \$s2 - \$s3$	three register operands
Transfer	load word	lw \$s1,100(\$s2)	$\$s1 = \text{Memory}[\$s2 + 100]$	Data from memory to register
	store word	sw \$s1,100(\$s2)	$\text{Memory}[\$s2 + 100] = \$s1$	Data from register to memory
	and	and \$s1,\$s2,\$s3	$\$s1 = \$s2 \& \$s3$	three reg. operands; bit-by-bit AND
	or	or \$s1,\$s2,\$s3	$\$s1 = \$s2 \$s3$	three reg. operands; bit-by-bit OR
	nor	nor \$s1,\$s2,\$s3	$\$s1 = \sim (\$s2 \$s3)$	three reg. operands; bit-by-bit NOR
	and immediate	andi \$s1,\$s2,100	$\$s1 = \$s2 \& 100$	Bit-by-bit AND reg with constant
	or immediate	ori \$s1,\$s2,100	$\$s1 = \$s2 100$	Bit-by-bit OR reg with constant
	shift left logical	sll \$s1,\$s2,10	$\$s1 = \$s2 \ll 10$	Shift left by constant
	shift right logical	srl \$s1,\$s2,10	$\$s1 = \$s2 \gg 10$	Shift right by constant
Conditional branch	branch on equal	beq \$s1,\$s2,L	if ($\$s1 == \$s2$) go to L	Equal test and branch
	branch on not equal	bne \$s1,\$s2,L	if ($\$s1 != \$s2$) go to L	Not equal test and branch
	set on less than	slt \$s1,\$s2,\$s3	if ($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$	Compare less than; used with beq, bne
	set on less than immediate	slti \$s1,\$s2,100	if ($\$s2 < 100$) $\$s1 = 1$; else $\$s1 = 0$	Compare less than immediate; used with beq, bne
Unconditional jump	jump	j L	go to L	Jump to target address
	jump register	jr \$ra	go to \$ra	For procedure return
	jump and link	jalt L	$\$ra = PC + 4$; go to L	For procedure call

Logical And Operation in MIPS



```
int a = 12;    // Binary: 1100
int b = 10;    // Binary: 1010
int result;
result = a & b; // Bitwise AND: 1000 (decimal 8)
```

Assuming a= \$s1, b= \$s2, result= \$s3

MIPS Code:

and \$s3, \$s1, \$s2

Logical Or Operation in MIPS



```
int a = 12;    // Binary: 1100
int b = 10;    // Binary: 1010
int result;
result = a | b; // Bitwise or: 1110
```

Assuming a= \$s1, b= \$s2, result= \$s3

MIPS Code:

or \$s3, \$s1, \$s2

Logical NOR Operation in MIPS



```
int a = 12;    // Binary: 1100
int b = 10;    // Binary: 1010
int result;
result = ~ (a | b); // Bitwise Nor: 0001
```

Assuming a= \$s1, b= \$s2, c= \$s3

MIPS Code:

nor \$s3, \$s1, \$s2

How to find the complement of a value in MIPS?



$x = 10101100$

$0 = 00000000$

$x \mid 0 = 10101100$

$\sim(x \mid 0) = 01010011$ ← this is the bitwise complement of x

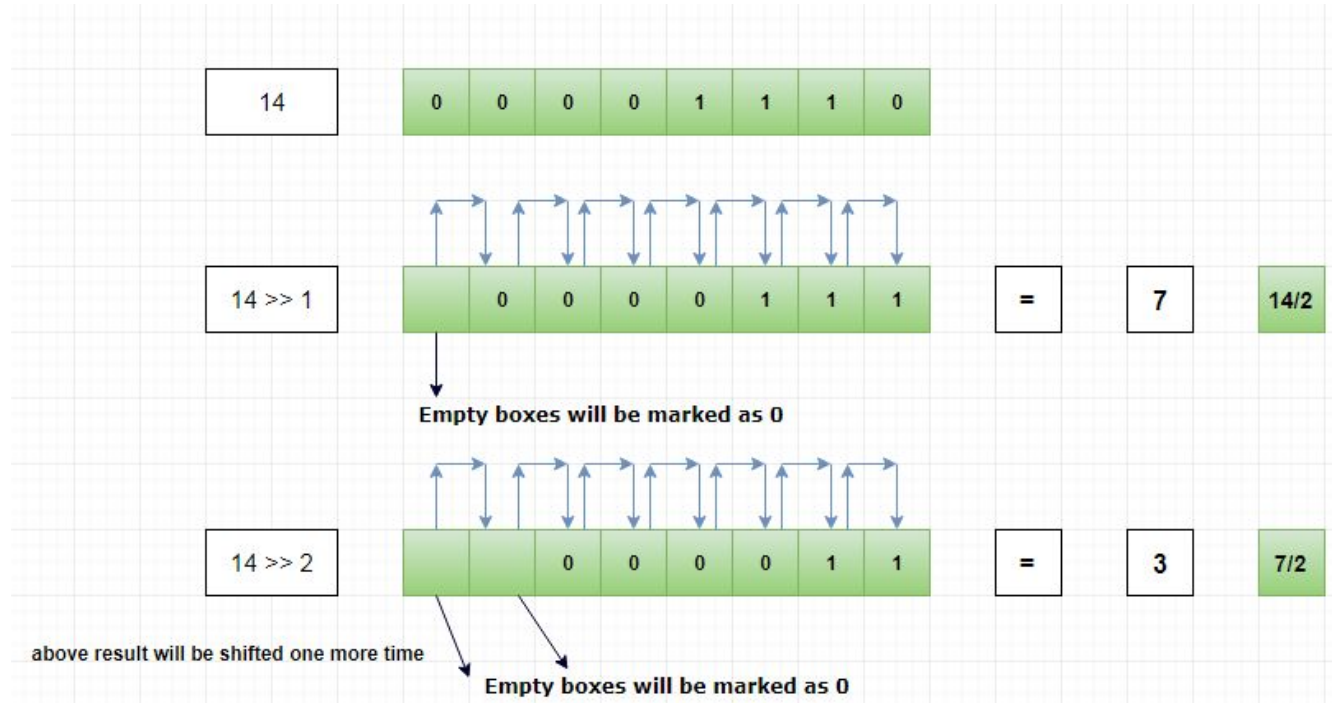
And Intermediate/ Or Intermediate



```
int a = 1023;  
int and_result = a & 255;  
int or_result = a | 256;
```

```
andi $t1, $t0, 255 # and_result = a & 255  
ori $t2, $t0, 256 # or_result = a | 256
```

Shift Operations



Left Shift and Right Shift Code



```
int a = 5;  
int left = a << 2;  
int right = a >> 1;
```

```
sll $t1, $t0, 2  # left = a << 2  
srl $t2, $t0, 1  # right = a >> 1
```

Modulus in MIPS



```
int a = 5;
```

```
Int c = a % 16; -> a - ((a >> 4) << 4) = 5 - 0 = 5
```

```
SO a % 2^n = a - ((a >> n) << n)
```

Assuming a= \$s0, c= \$s1

MIPS Conversion:

srl \$t0, \$s0, 4

sll \$t1, \$t0, 4

sub \$s1, \$t0, \$t1

Practice Problem



Find the MIPS Assembly Code for the following C Code:

```
int x = 5, y = 3, z = 7;  
int a = ~(x | y);    // NOR operation  
int b = (x & z) << 2; // AND followed by left shift  
int c = y | (z << 1); // OR with shifted value
```

Practice Problem



Find the MIPS Assembly Code for the following C Code:

```
int x = 5, y = 3, z = 7;
```

```
int a = ~(x | y);    // NOR operation
```

```
int b = (x & z) << 2; // AND followed by left shift
```

```
int c = y | (z << 1); // OR with shifted value
```

Assume:

x -> \$t0 = 5

y -> \$t1 = 3

z -> \$t2 = 7

a -> \$t3

b -> \$t4

c -> \$t5

Practice Problem



Find the MIPS Assembly Code for the following C Code:

```
int x = 5, y = 3, z = 7;
```

```
int a = ~(x | y);    // NOR operation
```

```
int b = (x & z) << 2; // AND followed by left shift
```

```
int c = y | (z << 1); // OR with shifted value
```

```
nor $t3, $t0, $t1
```

Assume:

x -> \$t0 = 5

y -> \$t1 = 3

z -> \$t2 = 7

a -> \$t3

b -> \$t4

c -> \$t5

Practice Problem



Find the MIPS Assembly Code for the following C Code:

```
int x = 5, y = 3, z = 7;
```

```
int a = ~(x | y);    // NOR operation
```

```
int b = (x & 24) << 2; // AND followed by left shift
```

```
int c = y | (z << 1); // OR with shifted value
```

Assume:

x -> \$t0 = 5

y -> \$t1 = 3

z -> \$t2 = 7

a -> \$t3

b -> \$t4

c -> \$t5

```
andi $t7, $t0, 24 # $t7 = x & z
```

```
sll $t4, $t7, 2  # b = $t7 << 2
```

How to Load Values in MIPS?



Int a=5;

Assuming a is in register \$s5

addi \$s5, \$zero, 5

Byte Addressable Memory



Byte Addressable Memory

Each Register in MIPS holds one index of the array which is of 32 bits (4 x 8) , 4 bytes

	<i>4-bytes wide</i>
@0x1000	1
@0x1004	2
@0x1008	3
@0x100C	4
@0x1010	5

Alias created –
'array_of_int'



Byte Addressable Memory



Each Register in MIPS holds one index of the array which is of 32 bits (4 x 8) , 4 bytes

	<i>4-bytes wide</i>
@0x1000	1
@0x1004	2
@0x1008	3
@0x100C	4
@0x1010	5

Alias created –
'array_of_int'

Value	Offset		Base Value
0			0
1	1	$0 + 4 \times 1$	

Byte Addressable Memory



Each Register in MIPS holds one index of the array which is of 32 bits (4 x 8) , 4 bytes

	<i>4-bytes wide</i>
@0x1000	1
@0x1004	2
@0x1008	3
@0x100C	4
@0x1010	5

Alias created –
'array_of_int'

Value	Offset		Base Value
0			0
1	1	$0 + 4 \times 1$	
4	4	$0 + 4 \times 4$	

Byte Addressable Memory



Each Register in MIPS holds one index of the array which is of 32 bits (4 x 8) , 4 bytes

	4-bytes wide
@0x1000	1
@0x1004	2
@0x1008	3
@0x100C	4
@0x1010	5

Alias created –
'array_of_int'

Value	Offset		Base Value
0 = \$s0			0
1	1	$\$s0 + 4 \times 1$	
4	4	$\$s0 + 4 \times 4$	

MIPS Assembly Language



Assume variable `h` is associated with register `$s2` and the base address of the array `A` is in `$s3`. What is the MIPS assembly code for the C assignment statement below?

```
A[12] = h + A[8];
```

```
lw    $t0,32($s3)    # Temporary reg $t0 gets A[8]
```

```
add   $t0,$s2,$t0    # Temporary reg $t0 gets h + A[8]
```

```
sw    $t0,48($s3)    # Stores h + A[8] back into A[12]
```

Syntax for Load



Data is loaded onto the register whenever we want to process the value of any variable.

```
A[12] = h + A[8];
```

Syntax for Load:

Operation Type	Register Name where we want to load	Offset (Base address if Register)

Syntax for Load



Data is loaded onto the register whenever we want to process the value of any variable.

```
A[12] = h + A[8];
```

Syntax for Load:

Operation Type	Register Name where we want to load	Offset (Base address if Register)
lw	\$t0	8 x 4 (\$s3)

Syntax for addition



Syntax for add

```
A[12] = h + A[8];
```

Operation Type	Destination	Operand 1	Operand 2
add	\$t0	\$s2	\$t0

Syntax for Store



Store function is used when we want to store the result in any register.

```
A[12] = h + A[8];
```

Syntax for storing:

Operation Type	Register from which value is collected	Destination Offset (base address)
----------------	--	-----------------------------------

Syntax for Store



Store function is used when we want to store the result in any register.

```
A[12] = h + A[8];
```

Syntax for storing:

Operation Type	Register from which value is collected	Destination Offset (base address)
sw	\$t	12 x 4(\$s3)

MIPS Practice Problem



Try following C codes:

```
h = 230;
```

```
A[10] = h + B[i];
```

Suppose, Compiler will assign A (base address) into \$s0, B(base address) into \$s1, h into \$s2 and i into \$s3.

```
addi $s2,$zero, 230
```

MIPS Practice Problem



Try following C codes:

```
h = 230;
```

```
A[10] = h + B[i];
```

Suppose, Compiler will assign A (base address) into \$s0, B(base address) into \$s1, h into \$s2 and i into \$s3.

Cannot write `i*4($s1)`

Use Left shift to update value of i

MIPS Practice Problem



Try following C codes:

`h = 230;`

`A[10] = h + B[i];`

Code to Load B[i]

`addi $t0, $zero, 230      # h = 230`

`sll $t2, $t1, 2          # $t2 = i * 4 (word offset for B[i])`

`add $t3, $s1, $t2        # $t3 = address of B[i]`

`lw $t4, 0($t3)          # $t4 = B[i]`

MIPS Practice Problem



Try following C codes:

```
h = 255;
```

```
A[j] = h + B[i];
```

Suppose, Compiler will assign A (base address) into \$s0, B(base address) into \$s1, h into \$s2 and i into \$s3 and j into \$s4



Thank you!